JUNTANDO TUDO PIPELINES DE DADOS

*Onze da noite, resposta a incidente: quais endereços bateram no `/admin` mais de vinte vezes na hora anterior, e com quais agentes de usuário? Noventa arquivos comprimidos, **quarenta gigabytes**. A primeira tentativa rodou **três horas** e devolveu uma lista errada por quatro motivos, nenhum deles um erro visível. O pipeline correto roda em **quatro minutos** e cabe em cinco linhas — e usa os mesmos comandos. **A diferença não é vocabulário; é método.***

070
GUIAS LINUX

OS SETE ESTÁGIOS — DESCARTE CEDO, ESTRUTURE TARDE

1	2	3	4	5	6	7
LOCALIZAR quais arquivos	**REDUZIR** descartar o que não interessa	**EXTRAIR** isolar os campos	**NORMALIZAR** uniformizar a forma	**AGREGAR** contar, somar, agrupar	**ORDENAR** o relevante no topo	**APRESENTAR** formatar para ler
`find · zcat · ls`	`grep -F · -E`	`awk · cut · grep -o`	`tr · sed · awk`	`awk · uniq -c`	`sort -rn · -k`	`head · column · jq`
os únicos que reduzem VOLUME		dão estrutura ao que sobrou		resumem em resposta		entrega

A ordem não é decorativa. Tudo depois do estágio 2 paga o preço do que passou: um `grep` caro antes de um barato faz o caro processar cem vezes mais linhas do que precisava — foi um dos quatro erros da abertura. **Filtre por arquivo antes de filtrar por linha, filtre por linha antes de extrair campo, e só estruture o que sobreviveu.** Nem todo pipeline tem os sete: um relatório rápido é `grep | sort | uniq -c | sort -rn`, ou seja, os estágios 2, 5 e 6. Mas quando o resultado sai errado, a pergunta **"em qual estágio isso se perdeu?"** encurta a depuração de uma hora para dois minutos, porque cada estágio é testável isoladamente. Os outros três erros daquela noite: o `grep admin` casou `/administrativo` e `admin.css` (faltou ancorar na requisição); o `>>` dentro de um laço abriu o arquivo noventa vezes e **grudou a última linha de um arquivo na primeira do seguinte**; e metade da pergunta — os agentes de usuário — simplesmente ficou de fora.

CONSTRUIR POR ESTÁGIO · E O QUE REALMENTE CUSTA TEMPO

kali@kali: ~/lab-pipeline

```
# 1. veja como os dados REALMENTE são
$ zcat acesso.log.gz | head -3
# 2. um estágio por vez, com head na frente: instantâneo
$ zcat acesso.log.gz | head -1000 | grep -F ':02:' | head -3
# 3. confira a CONTAGEM a cada estágio — é onde a perda aparece
$ zcat acesso.log.gz | wc -l
200000
$ zcat acesso.log.gz | grep -cF ':02:'
66701
# 4. o pipeline final: 4 minutos sobre 40 GB
$ LC_ALL=C zcat /var/log/nginx/acesso-*.gz \
    | grep -F ':02:' \
    | grep -E '"(GET|POST) /admin(/|\?| )' \
    | awk -F'"' '{split($1,c," "); print c[1] "\t" $6}' \
    | sort | uniq -c | sort -rn | head -20 | column -t -s $'\t'
```

Para inspecionar um pipeline já pronto, o `tee` grava o meio sem interromper o fluxo: `… | tee /tmp/e2.txt | awk … | tee /tmp/e3.txt | sort -rn`, e depois `wc -l /tmp/e[23].txt`. Para saber se um comando longo **está andando ou travado**, `pv enorme.log | grep -F 'ERRO' | wc -l` mostra vazão e progresso **sem alterar os dados**.

DESEMPENHO: QUATRO DECISÕES, QUASE TODO O TEMPO

`barato antes`	`grep -F` > `-E` > `-P` > `awk` com condição composta. Encadeados, **o barato vem primeiro** — inverter custa 10×
`uma passada`	`awk '/ERRO/{n[$1]++} END{…}'` substitui `grep|cut|sort|uniq -c`: 2 processos em vez de 4, e de 2 a 5× mais rápido **em arquivo grande com poucas chaves** (em volume pequeno, a cadeia legível vence; e o `sort` externo é imbatível quando as chaves não cabem na memória)
`LC_ALL=C`	vale para `grep`, `sort` e `awk` — e muda **resultado**, não só velocidade
`xargs -P 8`	só ajuda quando o trabalho **se divide por unidade independente** e o gargalo é CPU. Em HD mecânico, oito processos disputando cabeçote são **mais lentos** que um

OS ERROS QUE NÃO APARECEM COMO ERRO

`uniq sem sort`	conta em silêncio errado — e `join` sobre entrada não ordenada perde pares
`xargs -P`	**intercala a saída** dos processos: grave um arquivo por processo, ou use `parallel`, que serializa. **A redução final é sempre sequencial**
`nome com espaço`	quebra `for f in $(ls)` e `xargs` sem `-0`: use `find … -print0 | xargs -0`
`sem \n final`	`>>` gruda registros. Na dúvida, `tail -c 1 arquivo | xxd`
`\r do Windows`	campeão de falha silenciosa — `cat -A` mostra `^M`, `^I` e `$`
`entrada vazia`	produz nada e sai com **status 0**: indistinguível de "processou tudo e não achou". Se importa, conte antes
`subshell`	`cat x | while read l; do ((n++)); done` deixa `n` em zero — use `done < <(cat x)`
`SIGPIPE`	`| head -5` mata o produtor, o que é **correto** — mas sob `set -o pipefail` vira "falha" do pipeline

Pipeline que se repete vira script: `#!/usr/bin/env bash`, `set -euo pipefail`, parâmetros nomeados com mensagem de uso (`"${1:?uso: $0 <dir> <hora>}"`), limite configurável por variável, `export LC_ALL=C` e o `--` antes dos caminhos. **E saiba quando parar:** junção de várias tabelas grandes é banco de dados (`duckdb` lê CSV e Parquet direto do disco); estrutura profunda, estado complexo entre registros, tratamento de erro por registro ou necessidade de **testes** pedem um programa. A heurística honesta: **mais de cinco ou seis estágios, mais de um `awk` não trivial, ou você vai ter de explicá-lo a alguém na semana que vem** — escreva em Python. Não é derrota: é a mesma escolha que fez o `awk` substituir o `cut`.

Paralelizar por alvo multiplica o tráfego que você gera no alvo. `xargs -P 8` sobre uma lista de hosts gera **oito vezes** o volume — e o contrato de um teste autorizado costuma ter limite de taxa. Estourá-lo é **indisponibilidade provocada**, mesmo sem intenção e mesmo com autorização para testar (Lei 12.737/2012). Use `-P` compatível com o combinado, `parallel --delay` para espaçar, e **prefira paralelizar o processamento local** dos resultados, que não toca o alvo. Pelo mesmo motivo, **todo pipeline que gera lista de alvos passa por um filtro de escopo** — como **estágio obrigatório**, nunca como conferência mental: `awk 'FNR==NR{ok[$1];next} $1 in ok' escopo.txt candidatos.txt` é o que impede que um `/16` digitado errado vire varredura fora do escopo.

RESUMO RÁPIDO

- ✓ **Todo pipeline tem sete estágios:** localizar, reduzir, extrair, normalizar, agregar, ordenar, apresentar. Nomeá-los é o que permite perguntar "em qual estágio isso se perdeu?".
- ✓ **Descarte cedo, estruture tarde.** Só os dois primeiros reduzem volume; filtro barato antes de filtro caro.
- ✓ **Construa um estágio por vez, com `head` na frente**, e confira a contagem depois de cada um — é onde as perdas silenciosas aparecem.
- ✓ **Uma passada de `awk` costuma vencer `grep|cut|sort|uniq -c`** em arquivo grande com poucas chaves; em volume pequeno, a cadeia legível vence.
- ✓ **`LC_ALL=C` muda desempenho e resultado.** Fixe em qualquer script cuja saída seja ordenada ou comparada.
- ✓ **Paralelize por unidade independente e cuide da saída:** `xargs -P` intercala; a redução final é sempre sequencial.
- ✓ **Os erros que não aparecem como erro:** `uniq` sem `sort`, nome com espaço sem `-print0`, `\r` do Windows, entrada vazia com status 0, variável perdida em subshell, `SIGPIPE` sob `pipefail`.
- ✓ **Pipeline que se repete vira script** com parâmetros, uso, `set -euo pipefail` e `--`.
- ✓ **Saiba quando parar:** junção de tabelas grandes, estrutura profunda, estado complexo ou testes = escreva o programa.

NO KALI

A triagem de resultado de varredura é exatamente um pipeline de sete estágios: localizar (`find . -name '*.gnmap'`), reduzir (`grep -F 'Ports:'`), extrair (`awk`), normalizar (`tr -s ' '`), agregar (contagem por porta), ordenar e apresentar (`column -t`). Escrever isso como **script parametrizado**, e não como linha decorada digitada de novo a cada teste, é o que permite rodar a mesma triagem em todo engajamento sem reconstruir o raciocínio às onze da noite. **O `tee` preserva a evidência bruta**, e isso importa juridicamente: numa resposta a incidente ou num teste com relatório, o dado intermediário é **prova**. `zcat *.gz | tee bruto-preservado.txt | grep …` guarda a entrada exata do processamento, e um `sha256sum` sobre ela fixa a **cadeia de custódia** — reprocessar depois com o pipeline corrigido só é possível se o bruto foi guardado. **A redação de dado sensível é o último estágio, e é verificada:** antes de entregar qualquer arquivo gerado, passe o `sed` / `jq` de redação e **depois confira com `grep -c` procurando o que deveria ter sumido** — um pipeline de redação que falha em silêncio entrega senha em claro num PDF de relatório. E lembre que **o ambiente muda o estágio 1**: numa máquina da família RHEL os logs de serviço estão no `journald` em formato binário, então o começo é `journalctl --since … -o json` em vez de `zcat /var/log/*.gz` — e a saída em JSON cai direto no `jq`. O `parallel` e o `pv` vêm com um `apt install` no Kali, mas exigem o EPEL no RHEL: num alvo, conte só com `xargs`.